SOTER: Guarding Black-box Inference Service for General Neural Networks at the Edge

***Abstract*—Tremendous AI applications empowered by deep learning inference are increasingly deployed on edge devices to provide an interactive user experience. This necessitates securely offloading the service provider's proprietary model from the cloud to the edge, and using a co-located accelerator (e.g., GPU) to speed up model inference locally. A pragmatic approach is to use Trusted Execution Environments (TEEs) to protect the offloaded model from the untrusted software stack. However, despite much effort, securely outsourcing inference computations from a TEE to a faster, yet untrusted GPU remains a fundamental challenge. Existing TEE-based solutions either run the entire model only in TEE or take a handcrafted approach to partition insensitive model layers to run in a GPU. Although the latter solution achieves better performance, it requires substantial AI expertise and only protects confidentiality for specific models.**

We propose SOTER, a new inference system that can securely outsource sensitive inference computations from a TEE to a co-located untrusted GPU automatically for general models. Our key idea stems from the observation that most heavyweight model operators generally have an *associativity* property. With *associativity*, SOTER automatically profiles and partitions a fraction of operators with *morphed* parameters to run in a GPU. SOTER conceals the execution of these morphed operators and on-demand recovers the real execution results in TEE. In addition, SOTER designs an *oblivious fingerprinting* technique to safely detect integrity breaches of outsourced operators. Based on an extensive evaluation of SOTER and state-of-the-art secure inference systems with all nine prevalent deep learning models, SOTER demonstrates an encouraging latency reduction by up to 22.6× over baseline systems on complex big models (e.g., Transformer) while providing both sufficient confidentiality guarantees and high inference accuracy. Meanwhile, evaluation results on model integrity attacks show that SOTER can quickly detect integrity breaches with high probability and zero false positives.

I. INTRODUCTION

Driven by the remarkable success of Deep Learning (DL) [17], [68], edge computing applications are trending to utilize Deep Neural Networks (DNNs) to help analyze real-time data by taking inputs (e.g., images) and transforming those inputs into actionable insights (i.e., inference). For instance, autopilot navigation [6], [10], industrial automation [24], speech recognition [39] and visual assistance [49] are pervasive examples powered by DNN models (in short, models). Traditionally, AI service providers host their proprietary models on the cloud and provide inference service interfaces to clients. To utilize the model, clients can send inference queries to the cloud and receive results through a web portal [68].

While traditional inference services are primarily hosted on the cloud, increasingly inference workloads are run natively on edge devices for two reasons. First, numerous edge computing applications require millisecond level responses to conduct real-time data analysis [22], [66], which makes a cloud-based approach unsuitable because clients have to transfer high volume of queries over the network [45]. Second, since clients' queries can be sensitive (e.g., images for face recognition), existing cloud-based approaches leverage homomorphic encryption [15], [21] and Garbled Circuits [4] to protect clients' data, coming at substantial computational overhead. For example, in Delphi [45], a state-of-the-art cloud-based secure inference system, making one inference query requires ∼60MB communication and ∼4s latency, violating the stringent latency requirement of real-time applications.

However, offloading inference services from the cloud to the edge poses a threat to model confidentiality. Specifically, in order to obtain a qualified model with high inference accuracy, the service provider needs substantial resource and efforts to design an optimal model architecture and train the model parameters with enormous training data [47]. If directly publishing an offloaded model with *white-box* accesses at the edge, a client corrupted by the adversary (e.g., a business competitor) can directly observe the proprietary model architecture and parameters [44], [70]. To this end, there is an urgent demand to protect the confidentiality of an offloaded model with *black-box* inference protocols. *Black-box* means that a client can query the model as an oracle but will learn nothing about the exact model architecture and parameters.

To ensure the offloaded model can be queried as a *black-box* oracle, a recent pragmatic approach is to leverage Intel SGX [9], a pervasively used Trusted Execution Environment (TEE) to host models at the edge. SGX provides code integrity and data confidentiality for enclaves, such that the offloaded model hosted in enclave can not be seen or tampered with from other untrusted software stack running on the client's platform [5], [46]. Moreover, by leveraging the attestation tool of SGX, service providers can perform remote attestation to the client and ensure that the correct model is safely running at the client device equipped with SGX [9].

Unfortunately, SGX is not a panacea to solve the problem. In order to meet the stringent latency requirement for model inference at the edge [22], [66], it is urgent to develop an SGX-based secure inference approach along with a co-located hardware accelerator (e.g., GPU) to speed up model inference locally. However, existing approaches are facing a fundamental confidentiality dilemma between *black-box* access and fast execution with a co-located accelerator: either an approach has to enclose all inference computations within enclave to guarantee

Network and Distributed Systems Security (NDSS) Symposium 2021
21-24 February 2021
ISBN 1-891562-66-5
https://dx.doi.org/10.14722/ndss.2021.23xxx
www.ndss-symposium.org

model confidentiality, or outsource part of sensitive models to the untrusted accelerator for faster inference computing. As a typical work, MLcapsule [25] uses a modular approach that hosts the entire model layers in enclaves to guarantee model confidentiality. Nevertheless, MLcapsule runs model inference purely with CPU and bans the choice of GPU acceleration, achieving unsatisfactory performance (up to 22.6% latency slowdown, shown in §V) for diverse inference workloads.

Confronted with the confidentiality dilemma, known remedies including eNNclave [54] and DeepEnclave [23] take a partition-based approach to leverage the strong computation power of a co-located GPU. Specifically, these approaches require manually partitioning the offloaded model into public and private layers. By holding the private layers in enclave and outsourcing public layers to GPU, sensitive parameters in the private layers remain confidential while the untrusted accelerator can handle the public layers fastly, achieving lower inference latency than pure CPU inference (i.e., MLcapsule).

Although existing partition-based solutions can speed up model inference by engaging the GPU power, but in general, a handcrafted partition approach is error-prone and not secure enough to support general DNN models. In more detail, for some specific models with simple and well-known architectures (e.g., ResNet [63] and VGG [53]), existing solutions can be error-free [54]. This is because, these simple models are proved to have a lucid sensitivity boundary among layers, i.e., using an explicit set of extraction layers (with sensitive parameters) to distill input features, and use other layers (with insensitive parameters) to perform class-specific transformation on intermediate results [62], [29], [71]. Hence, it is secure for these models to keep all explicit sensitive layers within enclave and partition other layers to GPU.

However, existing handcrafted model partition approaches are error-prone to big models. This is because, as models are becoming increasingly deep with the rapid advancement of AI techniques such as neural architecture search (NAS) [14], [7], DNN models are evolved to have complex architecture composed of numerous intricate inter-layer linkages [38], [74], [51], making a handcrafted model partition approach prone to identify the real sensitive layers and thus cause confidentiality leakage (shown in §IV-B).

Moreover, even with an error-free partition, existing partition-based systems still expose a large attack surface to an adversary outside enclave since all partitioned model operators in GPU are not under the protection of SGX. Concretely, even if the parameters of a partitioned operator in GPU is insensitive, an adversary outside enclave (e.g., a malicious OS) can wreck the integrity of partitioned operators by modifying the parameters or altering the intermediate inference results, misleading an offloaded model to produce wrong outputs to drop the inference accuracy and affecting the service quality.

To this end, we propose SOTER, the first partition-based approach that provides stronger confidentiality guarantees to general DNN models and conquers the inherent integrity threat of model partitioning. Our key weapon to achieve both goals is the general *associativity* property of DNN operators. *Associativity* means that the way that factors are grouped in an operators computation does not change the final result [43]. Since most computational-intensive DNN operators are associative (e.g., *Convolution*, *Linear*), SOTER can securely morph these associative operators' parameters with hidden factors, partition them to run in GPU, and recover the real inference result with factors in enclave without compromising confidentiality.

Based on that key weapon, SOTER carries a *Morph Then Recover* (MTR) protocol in enclave to securely partition heavyweight inference computations to GPU. Specifically, before the inference, SOTER profiles the entire model to learn the architecture and *associative* operators to find an optimal partition plan. Then, SOTER morphs the parameters of a fraction of *associative* operators with randomly generated *blinding coins* (non-zero unsigned integers) in enclave, safely partitions these *morphed* operators to GPU for faster inference computing, and then *recovers* the real inference result with reciprocal coins in enclave when needed. With the MTR protocol, all partitioned operators' parameters in GPU are hidden without revealing the exact value, while we proving that the original distribution of parameters has negligible influence on confidentiality (§IV-B).

To tackle the integrity challenge, SOTER dynamically derives *oblivious fingerprints* to detect integrity breaches outside enclave based on the same observation of parameter morphing (i.e., the associativity property). *Fingerprints* are tuples of inputs and their corresponding outputs for a given operator, which hold the ground-truth correctness. SOTER creates a set of fingerprints for each partitioned operator in a *pre-processing* phase where all inference computations are run in enclave to ensure no perturbation occurs. Then, in an *inference* phase, when an inference intermediate result (IR) switches from enclave to GPU for computing a partitioned operator O, SOTER verifies the integrity of O by sending a checking query (i.e., O's fingerprint input) along with the IR, and comparing the computed output with the expected O's output (i.e., O's fingerprint output) in enclave. Any mismatches indicate integrity breaches of the partitioned operator O outside enclave.

However, using only a static set of fingerprints to detect integrity breaches is not secure since the adversary can observe these fixed fingerprint inputs through long-term monitoring and evade the integrity checking. SOTER addresses this problem by making fingerprints oblivious. Specifically, with a fixed set of fingerprints after pre-processing, SOTER dynamically derives more oblivious fingerprints by randomly assembling several existing fingerprints with randomly generated coefficients in enclave. The newly generated fingerprints' inputs statistically share the same distribution as the normal IRs (§IV-D), thus the adversary outside enclave can not distinguish specific checking queries (i.e., fingerprint inputs) and evade our verification.

We implemented SOTER on the Slalom [64] codebase, a mature and lightweight framework for writing neural networks. We evaluated SOTER with nine state-of-the-art DNN models covering four prevalent types: Multi-layer Perception (MLP), Convolution Neural Network (CNN), Recurrent Neural Network (RNN) and Transformer. We compared SOTER to two state-of-the-art TEE-based secure inference systems (MLcapsule [25] and eNNclave [54]). Our evaluation shows that:

- **Security:** SOTER provided stronger confidentiality guarantees to general DNN models while existing handcrafted partition system (eNNclave) only protected specific models. Further, SOTER can quickly detect integrity breaches

of partitioned operators outside enclave with high probability and zero false positives (§IV-D).

- **Performance:** SOTER was efficient. SOTER's inference latency was 3.5×∼ 22.6× lower than the pure-TEE inference baseline (MLcapsule) and has up to 1.8× latency reduction over the handcrafted partition baseline (eNNclave) (§V-A).
- **Accuracy:** SOTER was accurate. SOTER can achieve the ground-truth accuracy of the original model (same as MLcapsule), while existing partition-based baseline (eNNclave) caused 1.1%∼6.3% accuracy drops (§V-B).

Our major contribution is the MTR protocol, the first work that provides stronger confidentiality and integrity guarantees to general DNN model inference at the edge. SOTER's security guarantees are two-fold: for confidentiality, SOTER can achieve the same guarantee as the handcrafted partition-based approach for all their evaluated simple models and SOTER provides stronger guarantee for complex big models (§IV-B); for integrity, SOTER can quickly detect integrity breaches with high probability while existing partition-based systems ignore this severe threat outside enclave (§IV-D). Furthermore, our extensive evaluation on prevalent models shows that SOTER can achieve low-latency while maintaining high-accuracy, whereas existing pure-TEE systems [25] cause latency explosion (§V-A) and existing handcrafted partition-based systems [54] incur prohibitive accuracy drops (§V-B). This makes SOTER unique to support mission-critical AI applications at the edge, such as home monitoring [12] and visual assistance [49].

In the remaining of this paper, §II introduces background. §III gives an overview of SOTER. §IV describes SOTER's design and security analysis. §V shows our evaluation. §?? discusses related work and §VII concludes.

II. Background and Related Work

A. Deep Neural Network

With the rapid advancement of deep learning, AI models powered by deep neural networks (DNNs) are increasingly deployed at the edge to serve mission-critical applications [6], [10]. As shown in Figure 1, a DNN model (in short, model) can be represented as a sequence of connected *layers* with each layer assigned a set of *operators*. An operator is either a *linear operator* or a *nonlinear operator* where a linear operator is weighted by a set of pre-trained *parameter matrices* (in short, parameters).

The life-cycle of a model can be divided into two phases: a training phase and an inference phase. In the training phase, to obtain an accurate model, the model provider has to design an optimal model architecture (e.g., through neural architecture search [14], [7]), passes the model with a private training dataset multiple times, and updates the numerical values of parameters by calculating the gradients with forward-backward propagations [47]. Both the model architecture and parameters are sensitive properties as they require substantial expertise knowledge and training efforts [60]. In the inference phase, a model takes input data and passes the data through all operators with fixed parameters. Without losing generality, we use image classification as an example to illustrate the model composition in next discussions.

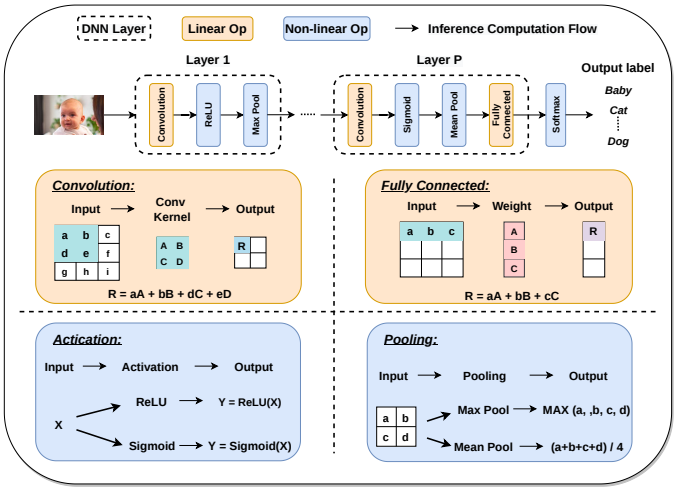


Fig. 1: An overview of a DNN model.

Inference workflow. Figure 1 shows the workflow of running an inference on a pre-trained model with pre-designed architecture and pre-trained parameters. A model M takes an input X (e.g., an image) and maps it to a corresponding output Y (e.g., a class label), which can be defined as a function $M : X \Rightarrow Y$. Specifically, an inference phase takes three steps. First, the model M passes the input X through all the operators and computes $Z(X)$ which is known as the *logits* [27]. Second, M utilizes the *softmax* function to normalize the values of $Z(X)$ and computes an output vector $V_i(X) = softmax(Z(X))$, where each element in $V_i(X)$ represents the probability of the input X belonging to class i while holding $V_i(X) \in (0, 1)$ and $\sum V_i(X) = 1$. Last, M computes and assigns the final class label $Y = argmax_i V_i(X)$ to the input X.

DNN operators. Operators are the key building blocks of a model where linear operators are proved to be the main computational bottleneck in inferences. For instance, for a classic VGG-19 inference on a single CPU thread, about 98% of the computation is spent on the linear operators [64].

Figure 1 shows two typical forms of linear operators: *convolution* and *fully-connected*. During the convolution computation, the convolution kernel (with its parameters) is placed at different locations of the input data. At each location, a sum of element-wise product is computed between the kernel and corresponding input values within the kernel window. The fully-connected operator conducts a weighted sum (i.e., a dot product) between an input and a flatten parameter vector which is owned by the fully-connected operator.

Nonlinear operators typically use *activation* functions, such as Rectified Linear Unit (*ReLU*): $f(x) = max\{0, x\}$ and *Sigmoid*: $f(x) = 1/(1+e^{-x})$ to approximate complex transformations [3], [72], and use *pooling* functions, such as the *max-pool* and *mean-pool* to reduce the dimension of intermediate computation results produced in inference computations [16].

Associativity. Suppose we have an input X and a non-zero number μ, a DNN operator F is associative if

$$(\mu^{-1} * \mu)F(X) = \mu^{-1}F(\mu X) \quad (1)$$

Typical linear operators (e.g., *convolution*, *fully-connected*) all have the associativity property because these operators conduct linear transformations on an input data. Take the

most commonly used, computation-expensive convolution operator [58] as an example. As shown in Figure 1, if we multiply each element in the convolution kernel by 2^{-1} (i.e., $\mu = 2^{-1}$), we will get the output $2^{-1}R$ while it always holds $(2 * 2^{-1}) * R = 2 * (2^{-1}R)$. Hence, the *convolution* operator is associative. This is applicable to the *fully-connected* operator as well.

Since nonlinear operators conduct nonlinear transformations on an input data, nonlinear operators usually do not have an associativity property (e.g., *Sigmoid*). However, under specific constraints, some nonlinear operators can also be associative. Specifically, when μ>0, the *ReLU* operator is associative because μ will not change the positive or negative of the input X, such that $\mu^{-1}max\{0, \mu X\}$ will get the correct output. The same constraint is applicable to the *max-pool* operator as well. Note that the *mean-pool* operator naturally has the associativity property because it actually conducts linear transformation (i.e., averaging) on an input data.

Operators that satisfy Equation 1 also meets a variant of the associativity property. Given that

$$\begin{cases} F(X_1) = Y_1, \\ F(X_2) = Y_2. \end{cases}$$

we have

$$F(\mu_1 X_1 + \mu_2 X_2) = \mu_1 Y_1 + \mu_2 Y_2 \quad (2)$$

SOTER leverages Equation (1) and Equation (2) as the key weapon to automatically partition general models without needing to identify security-sensitive operators in a handcrafted manner, which is error-prone. We illustrate our detailed designs in §IV.

B. Intel SGX

Intel Software Guard eXtension (SGX) [9] is a pervasively used hardware feature on commodity CPUs. SGX provides a secure execution environment called enclave, where data and code execution can not be seen or tampered with from outside. Code outside enclaves can enter an enclave by `ECalls`, and SGX uses remote attestations [9] to prove that a particular piece of code is running in an enclave on a genuine SGX-enabled CPU. SGX provides a trustworthy random source (`sgx_read_rand`), which invokes a pseudo-random generator through the RDRAND CPU instruction seeded by on-chip entropy sources [9]. Previous studies show that this random source complies with security and cryptographic standards and can not be seen or tampered with from outside enclave [5].

Recent work leverages SGX to guard black-box model inference at the edge. MLcapsule [25] encapsulates individual model layers and run inference computations purely inside the SGX. Such a modular approach makes it east to combine layers and form large networks but fails to provide fast inference speed owing to the computational bottleneck of the CPU.

Same as SOTER, eNNclave [54] and DeepEnclave [23] also take a partition approach to utilize a co-located GPU for low-latency inference, which is the dominant approach for secure inference at the edge. However, these two systems require manual effort to identify sensitive layers and partition the insensitive part from enclave to GPU. Such a handcrafted approach is only secure for specific models with lucid sensitivity boundaries among layers. Moreover, exiting partition-based approaches do not handle the inherent integrity threat after model partitioning, exposing a large attack surface to the adversary outside SGX enclave.

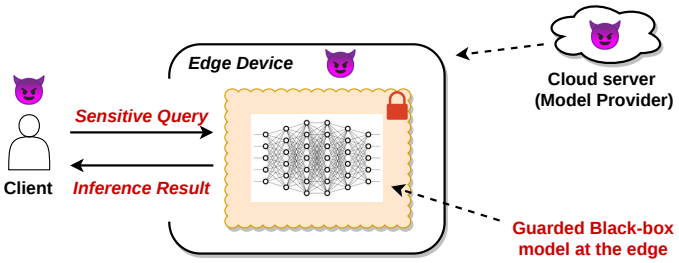


Fig. 2: The client-side inference scenario handled by SOTER.

III. OVERVIEW

A. System setup

We consider a client-side inference scenario shown in Figure 2. There are two parties in the system: the client and the model provider (or server). The client owns sensitive inference queries for a specific AI task. The server owns an appropriate model that has well-designed model architecture and well-trained parameters for the task. The server offloads its model to an SGX-equipped edge device. The offloaded model conducts client-side inference by taking client's sensitive queries, then sends the inference results back to the client.

B. Threat model and privacy goals

Similar to other SGX-based secure inference systems, namely the MLcapsule [25], eNNclave [54] and DeepEnclave [23], SOTER is designed for a two-party setting, where both parties have a bounded computational capability as in [73], [76], and try to learn information about the other parties' private data from the messages it receives. That is, the client C and server S will follow the inference protocol coded within SGX, but the client C wants to learn the model architecture and parameters and the server S attempts to learn the client's input data. Note that, different from prior work that assumes the offloaded model is built on well-known DNN architectures such as ResNet [63] and VGG-16/19 [8], [53], we intend to protect both the parameters and model architecture (number of layers, linkages between layers, etc) because recent advances in *neural architecture search* (NAS) has demonstrated that model architecture has a significant impact on model accuracy and has to be protected [14]. Since the two parties have conflicting interests, we do not assume two parties will collude to conduct attacks.

Note that, SOTER does not hide information that is revealed by the results of inference queries. This assumption stays in line with the threat model of other previous work, including both cryptographic-based and TEE-based secure inference systems [45], [32], [25], [54]. Protecting against such attacks that exploit the leakage of inference results is a complementary problem to that solved by SOTER. Indeed, many attacks have been successfully carried out even against systems that completely hide the model with black-box accesses [50], [69]. However, existing popular mitigations for these attacks, such

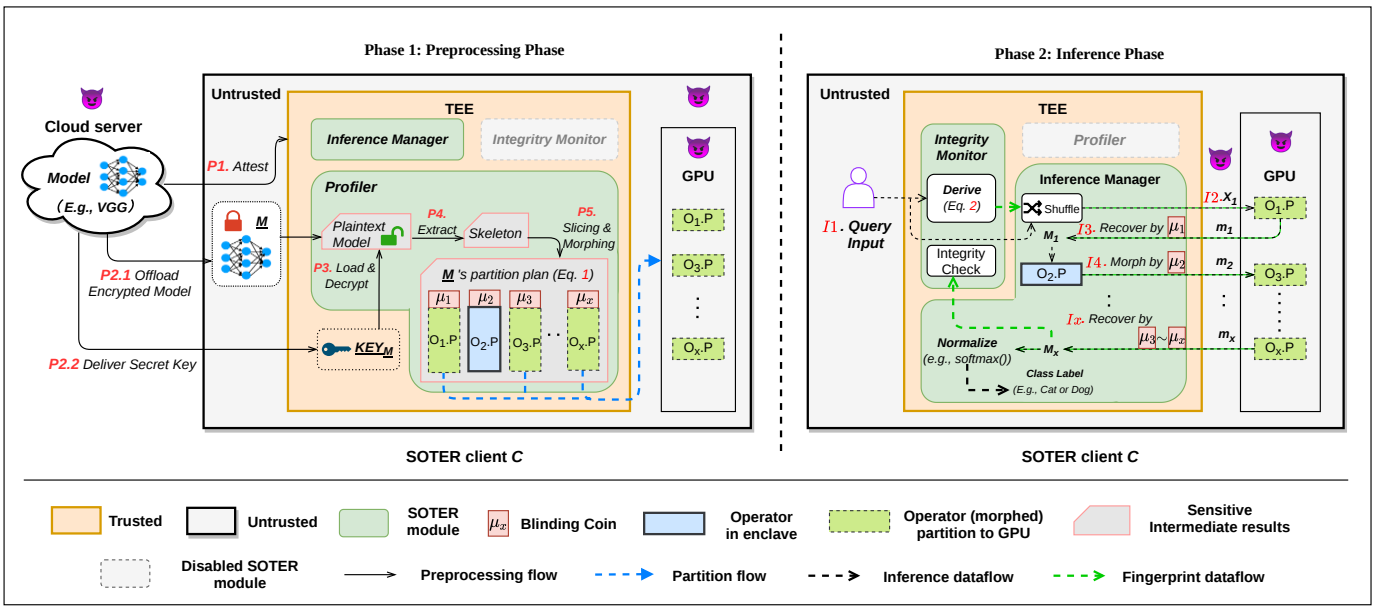


Fig. 3: The architecture of SOTER in a modular manner. SOTER's secure inference protocol has three modules running in the enclave (shielded in green). An adversary can observe and modify parameters of partitioned operators in the co-located GPU.

as differential privacy [13], can be combined with SOTER's protocol. We illustrate these attacks and mitigations in more detail in §VI.

SGX's threat model. SOTER has the same threat model for SGX as typical SGX-based systems [56], [67], [30]. We trust the hardware and firmware of Intel SGX, which ensures that code and data in enclave can not be seen or tampered with from outside. However, the client device could be under the control of a malicious OS or hypervisor (i.e., the adversary), and the adversary can observe and modify model parameters that is partitioned outside enclave. We trust that SGX's remote attestation service can identify genuine SGX devices from fake ones (e.g., emulated with QEMU). Same as DeepEnclave [23] and other SGX-based secure inference systems [25], [54], we do not intend to address the side-channel vulnerabilities against Intel SGX in this paper but we discuss the the potential vulnerabilities and mitigations in §VI.

C. System overview

SOTER is a secure inference system running at the client side. Different from a cloud-side system (e.g., Delphi [45]) that aims to protect the client's sensitive input data, in SOTER, the client's data is never transmitted to the untrusted cloud, hence SOTER naturally protects the client's data. Instead, SOTER focuses on protecting the offloaded model at the client-side. A client runs SOTER's protocols in a trusted module (i.e., the SGX enclave) and all system components outside enclave can behave arbitrarily.

SOTER's architecture. Figure 3 shows the architecture of a SOTER client. The client is equipped with an attested SGX enclave running only SOTER's inference protocols. Each SOTER client runs the MTR protocol in three trusted modules (in green) within enclave (in orange). The *profiler module* takes an offloaded encrypted model and its corresponding keys as input to extract the model skeleton (i.e., architecture), and automatically makes an optimal partition plan. The *inference manager module* takes the client's query input (i.e., data for inference) and manages inference computations according to the partition plan. The *integrity monitor module* dynamically derives fingerprint data flows with *oblivious fingerprints* that are indistinguishable to the client's inference data flows from an adversary's view outside enclave.

SOTER's two-phase design. At a high level, SOTER's protocol for secure inference at the client-side consists of two phases: an offline *preprocessing phase*, and an online *inference phase*. Specifically, the offline preprocessing phase is independent of the client's query input which changes regularly. We assume the offloaded model from the server is static, if the model changes, then both parties should re-run SOTER's preprocessing phase. After preprocessing, during the online inference phase, the client sends query input to get the eventual inference result by passing through SOTER's integrity monitor module and inference manager module. Note that, SOTER's two-phase design is most suitable for applications which could reserve sufficient preprocessing resources (see **Remark I**).

SOTER's workflow. In Figure 3, we show how SOTER leverages the general associativity property of DNN operators to automatically partition a model without identifying the sensitive (or insensitive) parts, which makes SOTER unique to provide strong confidentiality guarantees to general models. We also introduces how SOTER guards the entire inference process in two phases.

In the preprocessing phase: (**P1**) The server S attests to the SOTER client C to ensure legitimate SOTER's code is running on top of a genuine SGX-enabled hardware. (**P2.1**&**P2.2**) If the attestation fails, then the preprocessing phase aborts; Otherwise, after a successful attestation, S builds up a secure communication channel with C, then S offloads its encrypted model M to the untrusted memory outside enclave, and S also offloads the corresponding decryption keys KEY_M to

C's enclave. (**P3**) SOTER's trusted profiler module loads the encrypted model M in a layer-wise manner and decrypts M with KEY_M. Note that, for the latest version of SGX with sufficient memory support [57], we could directly load the offloaded model to enclave. (**P4**) With a plaintext model, SOTER extracts the model skeleton to capture the architecture information including the operator types, the number of layers, and the linkages between layers. (**P5**) With the architecture information, SOTER statically filters out all *associative* operators that meets Equation 1, and then invokes SOTER's MTR protocol to securely partition part of model operators to GPU for faster inference computing.

Specifically, in **P5**, SOTER's MTR protocol runs as follows: With a given security parameter θ, SOTER randomly selects $\theta\%$ associative operators, and runs SGX's pseudo-random number generator (PRNG) to produce (1) *fingerprints* of the selected operators for integrity checking (used in the inference phase), and (2) random positive unsigned integers for all operators. These numbers serve as the *blinding coins* to hide the operators' parameters [45] and are always kept secrete in enclave. Owing to the associativity property (Equation 1), SOTER morphs every selected associative operator by multiplying each element in the operator's parameter matrices with a blinding coin (the computation result can be recovered by using the reciprocal coin), and then partitions the selected operators with morphed parameters to GPU. In Figure 3, the selected operators (in green) are morphed with corresponding coins, e.g., the parameter of the third operator $O_3.P$ is morphed with a coin μ_3 and partitioned to GPU, while the second operator (in blue) is kept within enclave. In §IV-B, we illustrate our choice of θ by considering the *transferability* of a partitioned model, which is a widely used metric to evaluate a black-box model's confidentiality under adversarial attacks [76], [18], [42], [65].

Note that, we choose to run the preprocessing procedures (**P1**∼**P5**) at the client-side because a client C might select his own θ according to his GPU capability, e.g., choose a smaller θ if the GPU has limited usable memory, even if SOTER has chosen an appropriate θ for confidentiality.

In the inference phase, there are two types of data flows: the inference flow (black dotted line) and the fingerprint data flow (green dotted line). The life-cycle for the inference dataflow is: (**I1**) The client C sends a query input (i.e., data for inference) to the trusted inference manager module and shuffles with the fingerprint input (introduced below). (**I2**) The inference manager forwards C's input X_1 to the GPU, and the first *morphed* operator $O_1.P$ takes X_1 as input and computes the output m_1. (**I3**) Since the next operator $O_2.P$ is expected to run within enclave according to the plan made in the preprocessing phase, the inference manager takes the computation result of previous partitioned operators (i.e., m_1) as the input, recovers m_1 with all previous partitioned operators' blinding coins since the last kernel switch (in Figure 3's circumstance, SOTER recovers with only $O_1.P$'s blinding coin μ_1), and gets the recovered result M_1. (**I4**) After that, the second operator $O_2.P$ computes with M_1 and gets an intermediate result (IR), then morphs IR with the $O_2.P$'s blinding coin μ_2 and forwards the result m_2 to the partitioned operator $O_3.P$ in GPU.

The above procedures (**I2**∼**I4**) repeat until all partitioned operators in GPU are computed. In (**Ix**), with the last computation result m_x from $O_x.P$, SOTER recovers the real computation result by using all previous partitioned operators' blinding coins since the last kernel switch (same as the recover step in **I3**). In this case, SOTER recovers with $\mu_2 \sim \mu_x$ and gets the real inference result M_x. Then, SOTER runs a standard DNN normalization function (e.g., softmax()) on M_x to get the class label and returns the label to the client C.

TABLE I: SOTER's protocol variables.

Variable	Description
$\Theta(default = 0.8)$	Portion of associative operators partitioned to GPU.
O_g	The set of partitioned operators in GPU.
O_e	The set of operators maintained in enclave.
$O(O_g \cup O_e)$	The whole set of model M's operators.
$Para(O_i)$	Parameters of operator O_i.
FP_{O_i}	Fingeprint of operator O_i.
μ_i	Blinding coin of operator O_i.
$K(default = 10)$	The initial number of fingerprints for each $O_i \in O_g$.
$W(default = 2)$	Top-W operators are always maintained in enclave.

The fingerprint data flow is used to detect integrity breaches of partitioned operators outside enclave. Specifically, with a set of partitioned operators' fingerprints produced in the preprocessing phase, within the inference data flow, SOTER's trusted integrity monitor module dynamically derives oblivious fingerprints by utilizing the general associativity of Equation 2, injects these fingerprints into the inference data flow to check whether the parameters of partitioned operators (or the IRs) have been modified by the adversary outside enclave. We prove in §IV-D that the adversary outside enclave can not distinguish the fingerprint flow from the inference data flow, preventing the adversary from bypassing the integrity checking.

Remark I (SOTER's targeting applications). Same as other secure inference systems that involve two phases (Delphi [45], Gazelle [32]) (although they are designed for cloud-based secure inference), SOTER is best suited for applications whose inference is latency-sensitive, but is usually not performed frequently enough to take up all computational resources needed for preprocessing. Examples of such applications include speech-to-text systems like [19] and image classification systems like Google Lens [20].

IV. PROTOCOL DESCRIPTION

This section describes SOTER's MTR protocol for partitioning general DNN models with confidentiality and integrity guarantees. SOTER's MTR protocol utilizes the general associativity property of DNN operators to automatically profile the entire model (with Equation 1), randomly selects $\theta\%$ ready-to-partition operators to hide the model architecture, and then morphs these operators' parameters with hidden *blinding coins* in enclave to hide the plaintext value of parameters (§IV-A). We then quantitatively analyze the confidentiality guarantees of the MTR protocol, especially for discussing the selection of the security-sensitive parameter θ (§IV-B).

As mentioned in §III-B, a model partitioning approach has an inherent integrity threat to the adversary outside enclave. The adversary can observe and modify the partitioned operators' parameters or perturb the inference intermediate results (IRs). By leveraging a variant of associativity property (Equation 2), SOTER uses a set of *oblivious fingerprints* that holds the ground-truth inference results for each partitioned

operator to check the integrity (§IV-C), while ensuring the fingerprint data flow could share the same distribution as the IRs, such that the adversary can not distinguish the fingerprints and bypass our detection (§IV-D).

Similar to existing secure inference systems at the cloud-side [45], [32], we now give the formal definition of a partition-based secure inference system at the client-side as follows.

Definition I. *An inference system ψ running at the client C holds an offloaded model M with architecture α and parameters $(M_1, ..., M_l)$, and the client C holds input queries to run inferences on M. ψ is a secure inference system at the client-side if it satisfies the following guarantees:*

- **Security:**
  - **Confidentiality.** *We require that a curious client C does not learn anything about either the model M's architecture α or the plaintext parameters $(M_1, ..., M_l)$. Formally, with a partitioned model M_p under the view of C, we require ψ should guarantee a low transferability of M_p, which is defined as the ratio of the correctly computed inference examples that the partitioned model M_p can mimic the original model M (more details in §IV-B).*
  - **Integrity.** *Given that a client C could be corrupted or under the control of a malicious OS, which can observe and modify the parameters of partitioned operators or IRs outside enclave, we require that ψ has to quickly detect integrity breaches outside enclave with high probability and zero false positives.*
- **Performance.** *We require that without sacrificing security, ψ should partition as many computations as possible to a co-located accelerator (e.g., GPU) in order to meet the stringent latency requirements of modern applications at the client-side.*
- **Accuracy.** *Given a partitioned model M, on every input query X from client C, we require that the output returned by ψ should achieve exact the same inference result as $M(X)$, i.e., without any accuracy drops.*

A. Morph Then Recover (MTR) protocol

SOTER's MTR protocol is divided into two stages: a *morphing* stage in the preprocessing phase (handled by the profiler module), and a *recover* stage in the inference phase (handled by the inference manager module). In the morphing stage, SOTER first makes *blinding coins* for every operators (including both associative and non-associative operators), and then make a partition plan with a given security parameter θ. Then, in the recover stage, SOTER runs inference across GPU and enclave, and recovers inference results in enclave when needed. Algorithm 1 and Algorithm 2 show the MTR protocol.

Stage 1-1: parameter morphing with blinding coins. SOTER assigns every operator with a randomly generated unsigned integer, which serves as the blinding coin (in short, coin) to hide the plaintext value of partitioned operators' parameters. In our implementation, we use the trustworthy random source (`sgx_read_rand`) in SGX to generate coins and and assign coins to every operator in model M (Line 3∼8 in Algorithm 1).

SOTER's blinding coin design protects model confidentiality from two aspects. First, if an operator O_i is associative and is selected to be partitioned to GPU according to the profiling plan (introduced in **Stage 1-2**), then SOTER morphs $Para(O_i)$ by multiplying each element in $Para(O_i)$ with the corresponding coin μ_i. By hiding μ within enclave, SOTER ensures that an adversary outside enclave can not learn the real parameters before being morphed in enclave.

Algorithm 1: MTR protocol at client c (offline) (§IV-A).

```
▽ Preprocessing Phase (within enclave).
1  Initialization
2      μ ← ∅;
3  Function make_coin() do
4      foreach operator op in O do
5          index ← O.index(op);
6          coin ← ∅;
7          while coin = ∅||coin = sgx_read_rand() < 0 do
8              μ ← coin;
9  Function partition() do
10     foreach operator op in O do
11         op.partition ← False;
12         op.index ← O.index(op);
13         if op.index > W then
14             if 0 < normalize(sgx_read_rand()) < Θ then
15                 op.partition ← True;
16             if op is associative & op.partition then
17                 foreach element e in Para(op) do
18                     e ← μ_index × e ;          // Morph
19                 O_g.add(op) ;          // GPU operators
20             else
21                 O_e.add(op) ;          // TEE operators
22     O_g.copyTo ("cuda") ;          // Partition to GPU
```

Second, the blinding coin design ensures no confidentiality leakage will occur during kernel switches (IRs transferred between enclave and GPU). Consider a case in Figure 3 where O_1 (in GPU) is a linear operator and O_2 (in enclave) is a non-linear *ReLU* operator. If we only morph O_1 and do not assign a coin to O_2, then, with a client input X, O_1 outputs $y_1 = X(\mu_1 * O_1.P)$ and O_2 outputs $y_2 = ReLU(y_1/\mu_1) = ReLU(X * O_1.P)$. Since the *ReLU* operator just filters out negative values in parameter matrices and does not transform non-zero values, an adversary who observes both y_1 and y_2 will directly infer the value of coin μ_1, violating the confidentiality of the partitioned O_1 (concretely, $Para(O_i)$). However, since SOTER assigns coin μ_2 to O_2, O_2 outputs $y_2 = \mu_2 * ReLU(X * O_1.P)$, an adversary can observe only the value of $\mu_1\mu_2$ but can not infer μ_1 or μ_2 respectively.

Remark II (precise arithmetic). Using coins to hide parameters of partitioned operators requires multiplication of a parameter matrix by a secret number. As popular implementations of neural network inference perform arithmetic over 32-bit floating-point numbers [64], in order to morph parameters without losing precision, in our implementation we choose to generate 16-bit non-zero unsigned integers as coins which prevent multiplication values from growing exponentially and thus overflowing, proved by recent model compressing theories on half-precision floating-point numbers [40].

Stage 1-2: hidden architecture under selective partition. With a given security parameter θ, SOTER automatically partitions a portion of associative operators (that are morphed)

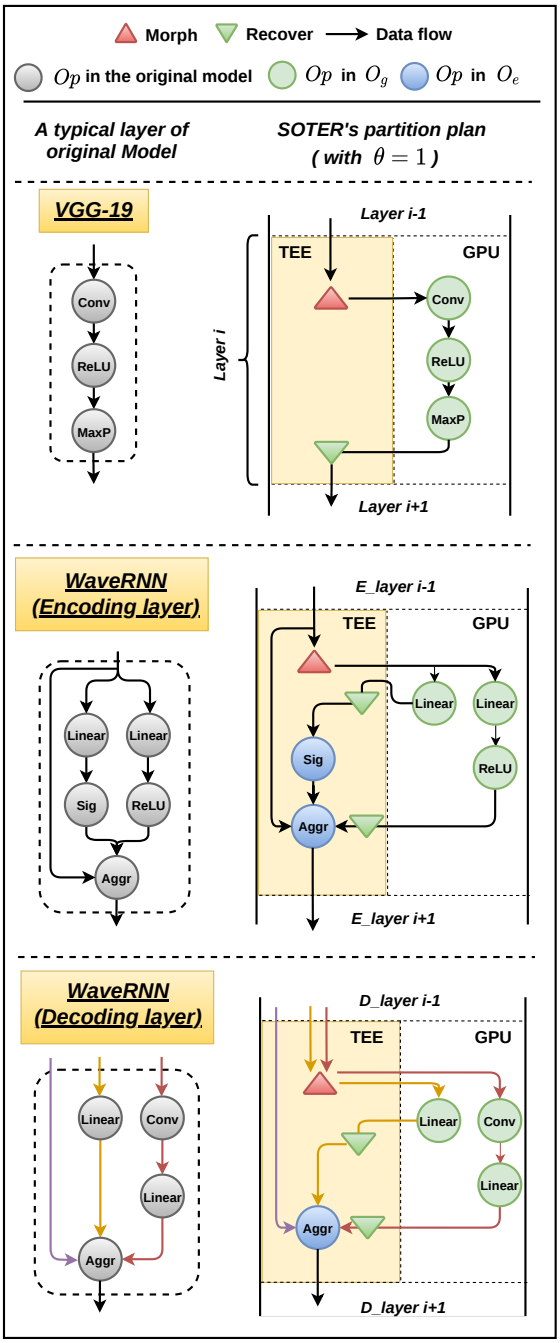


Fig. 4: SOTER's partition plan on two typical DNN models.

to GPU for inference acceleration. The function *partition()* in line 9∼22 of Algorithm 1 shows the pseudo code.

Specifically, in the preprocessing phase, SOTER iterates every operator in model M and randomly selects a portion of operators and adds them to the partition set O_g. Specifically, when meets an associate operator, SOTER uses the trustworthy source of SGX to generate a randomly number for the operator, and then normalize the number. If the normalized number is within the range of $0 \sim \theta$, then the operator is selected to join O_g. For instance, with $\theta = 0.8$, an associative operator would have an 80% chance to be partitioned to GPU. Then, after all operators are iterated, SOTER partitions all operators in O_g to

Algorithm 2: MTR protocol at client c (online) (§IV-A).

```
▽ Inference Phase (cross enclave & GPU).
1  Function secure_inference do
       ▽ I1.
2      every client input X do
3          X.copyTo ("enclave");
4          foreach index i ∈ range(W) do
5              X ← O_i(X) ; // Top-W TEE Inference.
6          start ← find the next op in O that belongs to O_g;
7          end ← find the next op in O that belongs to O_e;
8          while start < |O| do
               ▽ I2.
9              X.copyTo ("cuda");
10             foreach index i ∈ range(start, end) do
11                 X ← O_i(X) ;      // GPU Inference.
               ▽ I3.
12             X.copyTo ("enclave");
13             X ← ∏_{start}^{end-1} μ_i^{-1} × X ;      // Recover.
14             start ← end;
15             end ← start from O_end find the next op in O
                   that belongs to O_g;
16             foreach index i ∈ range(start, end − 1) do
17                 X ← O_i(X) ;      // TEE Inference.
               ▽ I4.
18             X ← ∏_{start}^{end-1} μ_i × X ;      // Morph.
19         X ← normalize(X);
20         return Top_1(X) ;      // Inference result.
```

GPU for inference acceleration.

Note that, SOTER will always hold top-W (W=2 by default) operators in enclave even if these operators are associative (Line 13). This design choice is made to ensure the input to the first partitioned operator in GPU (i.e., X_1 in Figure 3) is always oblivious, such that SOTER can obliviously check the integrity of partitioned operators by injecting fingerprint data flows (see more details in §IV-C).

We now give examples of running SOTER's MTR protocol to automatically partition neural networks (shown in Figure 4). We use two DNN models that achieve state-of-art performance on typical deep learning tasks: a VGG-19 model for computer vision task and a WaveRNN model for natural language processing task. VGG-19 is a popular CNN model known for its high-inference accuracy for image classification tasks [8]; WaveRNN is a state-of-the-art RNN model developed by Deepmind and Google for efficient speech-to-text transformations [33]. To illustrate our protocol in a lucid manner, in this example we choose $\theta = 1$, which means that all associative operators will be partitioned to GPU for acceleration.

The partition plans for typical layers of these two models are shown in Figure 4 (with $\theta = 1$). For VGG-19, SOTER can partition most of its operators to GPU because (1) VGG-19 is a typical layered model composed by associative operators including *convolution*, *ReLU* and *max-pool*, recalling that the two non-linear operators can also meet the associativity property given a positive coin (§II-A). (2) Most non-associative operators in VGG-19 such as *BatchNormal2d* and *Dropout* are de-activated throughout the inference process to avoid non-determinism [55]. For WaveRNN, we give partition plans for

both its encoding layer and decoding layer because WaveRNN is built by iteratively stacking these two typical layers and other scattered layers. We see that different from VGG-19, WaveRNN has more non-associative operators (e.g., *Sigmoid*) that can not be partitioned to GPU. This indicates that SOTER would achieve a more significant inference speedup when runs on convolution networks, which is proved in our evaluation §V.

Stage 2: black-boxed online inference. Now we present the end-to-end black-box inference process given that we have prepared coins and a portion of associative operators have been partitioned to GPU. As shown in Algorithm 2, SOTER first computes the inference result for the reserved top-W operators maintained in enclave. Then, SOTER iterates **I2**∼**I4** as follows. **I2**: SOTER finds the longest length of consecutively partitioned operators in O_g and computes the inference result by forwarding the input through these operators (line 6∼11). **I3**: Then SOTER copies the result back to enclave, recovers the real inference result by multiplying the reciprocal coins of all operators in last step (line 13), and then forwards the result to operators maintained in enclave (before the next copy to GPU). **I4**: SOTER morphs the forwarding result with all coins of enclave operators in last step (line 18), and then copies back to GPU. This procedure terminates when all partitioned operators in GPU are iterated. Then SOTER normalizes the final result and returns the result to the client.

B. Security analysis of MTR protocol

Attack assumptions. As we mentioned in (§III-B), we assume a client who is curious to learn the model offloaded from the server, and the server also tries to learn the client's private input. Since SOTER restricts data at the edge device and never uploads data to the cloud, we focus on analyzing how much information a curious client (the adversary) can learn in our security analysis. Specifically, we consider the adversary could be a business competitor whose purpose is to steal the offloaded model by using the partition information that he observes to train a *substitute model*, in order to reduce the competitive advantages of model owners.

Recall that we assume the adversary has a bounded computational capability as in [50], and does not know the whole training dataset for the offloaded model (otherwise it is totally unnecessary for the adversary to "*steal*" the model). This assumption stays in line with existing partition-based inference systems [76] to evaluate the confidentiality guarantee of a model's partition plan.

Evaluation setup. We evaluated the confidentiality guarantee of the MTR protocol with three CNN models, one RNN model and one transformer model. For the CNN models, we used VGG-19 [53], NASNet [75], and PNASNet [41], and pre-trained them on the widely used CIFAR-10 dataset [34]; For the RNN and transformer model, we used WaveRNN [33] and the official Nvidia transformer [2] and pre-trained it on the popular WNMT dataset [1]. With the bounded computational capability assumption, to stay in line with [76], [50], we isolated 90% of training data (e.g., 45,000 images in CIFAR-10), and the remaining 10% of training samples are used by the adversary. We assume the adversary can enrich his training dataset to train an *substitute model* by using data augmentation techniques [50].

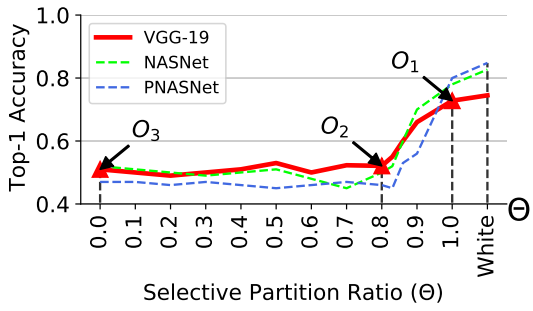


Fig. 5: SOTER's θ-transferability analysis of three CNN models on the adversarial substitute model SM.

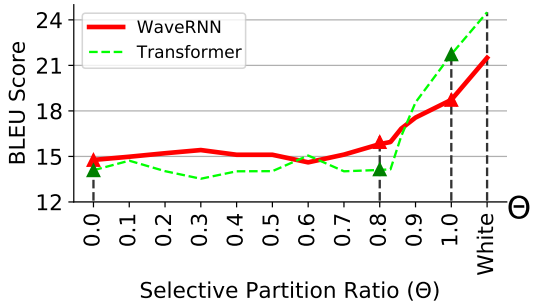


Fig. 6: SOTER's θ-transferability analysis of WaveRNN and transformer on the adversarial substitute model SM.

θ-transferability analysis. Recall our definition on secure model inference at the client-side (**Definition I**), we now analyze the confidentiality guarantee of SOTER's MTR protocol by evaluating the effects that the partition parameter θ could have on the quality of the adversarial substitute model SM trained by the adversary. With a given θ, we call a SOTER partitioned model outside enclave as $PM(\theta)$, whose architecture and parameters are completely observable by the adversary. The adversary then uses $PM(\theta)$ as the basic model (with both architecture and parameters) to train a substitute SM.

Same as prior work [76], we used SM's *θ-transferability* to evaluate the resultant security guarantee of SOTER's MTR protocol with a given partition parameter θ. Transferability means the portion of functionalities that SM can mimic the original offloaded model M. A higher transferability indicates that the SM trained by the adversary can mimic a larger portion of functionalities (i.e., more confidentiality loss), while a lower transferability reveals a smaller portion of functionalities are stolen (i.e., less confidentiality loss). Concretely, for CNN models, we use the *Top-1* accuracy as the transferability metric and we use the *BLEU score* for the RNN model, both of which are well-known AI metrics to measure the quality of a trained model.

Next, we first analyze the transferability of the typical CNN model VGG-19, and then we give analysis on other models. As shown is Figure 5, the red line (VGG-19) can achieve the ground-truth Top-1 accuracy (74.5%) when the X-axis is "*White*", i.e., a scenario where we placed the entire VGG-19 in GPU without activating SOTER's protocol. The ground-truth accuracy result is intuitive due to the white-box model being the same as the original model M. Next, we activated SOTER's MTR protocol to evaluate the effects that θ brings to Top-1 accuracy (i.e., transferability). When θ downgrades from 1 to 0,

we observed a significant accuracy drop from 71.8% to 54.3%.

Specifically, we collected three observations (**O1**∼**O3**) from our evaluation. **O1:** When $\theta = 1$, SM's accuracy decreases ∼2.7% because in that case the adversary can train SM with the knowledge of: (1) the partial architecture of model M (since operators used in VGG-19 are mostly associate, the adversary will get most of the architecture information when $\theta = 1$), and (2) morphed parameters of all associative operators partitioned to GPU. With these knowledge, the adversary trains SM with his training dataset and finally achieve a resultant substitute model SM with a relative high accuracy. **O2:** But when SOTER chooses $\theta = 0.8$, we observed a significant accuracy drop to 54.7%. This is because, SOTER only partitions an associate operator to GPU with 0.8 probability. Hence, we hide the architecture information of the model, and the adversary can only train SM with fragmented architecture information and morphed parameters of only a fraction of partitioned associate operators. As a result, the adversary can only train a low-accuracy model SM which can not substitute the original high-accuracy model M. **O3:** When θ is within the range from 0 to 0.8, we observed that the accuracy is almost steady without spikes. This is because, even with a smaller θ which means less associative operators are partitioned to GPU, the difficulty for the adversary to train a substitute model SM remains almost unchanged. Even when $\theta = 0$, which means no operators are partitioned to GPU (i.e., fully black-boxed), the adversary can still train from a blank model by using the adversary's augmented training dataset.

We also conducted the transferability analysis upon NASNet, PNASNet, WaveRNN and transformer, shown in Figure 5 and Figure 6. The similar downgrading trend and θ-*transferability* relationship reveal that hiding around 80% architecture information could be enough to guarantee sufficient model confidentiality. Hence, we choose $\theta = 0.8$ in SOTER by default, which obtains the maximum performance benefit (most associate operators would be partitioned to GPU for fast processing) when achieving comparable security level as fully black-boxed models (i.e., $\theta = 0$).

Comparison to baselines. Compared to SOTER, MLcapsule [25] runs the entire model within SGX at the cost of higher inference latency, but achieves the same strong confidentiality guarantee on offloaded models because all operators are protected by SGX, which corresponds to the $\theta = 0$ scenario in SOTER. Another baseline eNNclave [54] uses a handcrafted approach to manually identify sensitive (insensitive) model layers and partition insensitive layers to run in a GPU, which only provides confidentiality guarantee to simple CNN models with lucid sensitivity boundaries between layers (e.g., VGG series CNN models). We analyzed all evaluated models in eNNclave paper (i.e., VGG-16/19) and we found that in these simple models, SOTER's automatic partition approach can still achieve comparable transferability (with SOTER's $\theta = 0.8$), which implies that SOTER's MTR protocol that leverages the associativity property can provide sufficient confidentiality guarantees to general DNN models.

C. Detecting integrity breaches with oblivious fingerprints

In order to detect integrity breaches of partitioned operators outside enclave, SOTER's starting point is to use a simple but effective re-compute technique. The main concept of this idea is to generate checking queries in enclave at runtime, inject these checking queries to the normal inference data flow to go through the entire computing process of partitioned operators, and finally compare the returned inference results of checking queries with expected outputs in enclave. SOTER will report an integrity breach to the client if the inference result differs from the expected output.

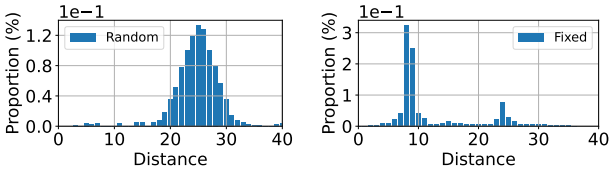
Algorithm 3: Fingerprint preparing (offline) (§IV-C).

```
▽ Preprocessing Phase (within enclave).
1 Function preparing() do
2     foreach k in range(0, K) do
3         input ← generate_random_input(M);
4         FP_0[k].input ← input;
5         FP_0[k].output ← O_0(input);
6         foreach operator O_i in O_g[1, |O_g|] do
7             FP_i[k] ← ⟨input : FP_{i-1}[k].output, output :
                O_i(FP_{i-1}[k].output)⟩
```

Unfortunately, since the adversary can persistently monitor the inference process and observe every intermediate result (IR) transferred from enclave to GPU, if using only a static set of checking queries, the adversary can easily distinguish those fixed checking queries among all IRs, hence bypassing SOTER's integrity check by computing and returning the correct inference result to the enclave. However, if we trivially use different checking queries for every integrity check (such that the adversary can not determine the fixed checking queries), we have to generate different checking query input and precompute an operator's expected output for every checking query input, which is extremely costly as we have to run slow CPU inference computing in enclave for each partitioned operator and for each client's inference query.

SOTER tackles this challenge by making all checking queries indistinguishable (oblivious) from other normal inference queries, and efficiently generating those oblivious checking queries without needing to pre-compute the expected checking outputs in enclave. Specifically, SOTER's oblivious fingerprinting technique can be divided into two steps. First, SOTER makes a set of *cornerstone fingerprints* for each partitioned operators during the preprocessing phase, which serve as the basis of checking queries. Second, during the inference phase, SOTER dynamically derives oblivious fingerprints from existing cornerstone fingerprints prepared in the preprocessing phase. Owing to the variant of the associativity probability (Equation 2), SOTER can skip the step of pre-computing the expected checking outputs in enclave before sending these checking queries to GPU. There are two critical steps:

Step 1: preparing cornerstone fingerprints. SOTER prepares cornerstone fingerprints after a partition plan is made and before a fraction of operators are partitioned to GPU (i.e., between line 21∼22 of Algorithm 1). Algorithm 3 shows the preparing procedure. SOTER prepares K (by default K=10) cornerstone fingerprints for each partitioned operator during the processing phase. A fingerprint is a two-element tuple: $\{input, output\}$, where the input is randomly generated in enclave (line 3) and the output is computed by forwarding the input through a ready-to-partition operator (line 5). The whole



(a) Oblivious fingerprint. (b) Fixed fingerprint.

Fig. 7: Distribution of L2 distances between randomly sampled intermediate results.

preparing procedure is running within enclave. By doing so, we ensure the correctness of each cornerstone fingerprint's output, Now we hold K correct input-output tuples (i.e., fingerprints) for each partitioned operator.

Step 2: dynamically derive oblivious fingerprints. SOTER dynamically derives oblivious fingerprints at runtime without needing to pre-compute expected outputs for each partitioned operator. Specifically, with a fixed set of cornerstone fingerprints $FP_O[K]$ for each partitioned operator O, SOTER randomly selects T fingerprints and generates T random numbers for each fingerprint by using SGX's trustworthy random number generator. Then, SOTER applies the associativity property (i.e., Equation 2) to generate a new fingerprint FP_{New} by using the T cornerstone fingerprints and random numbers. Concretely, the input for FP_{New} is $\sum_{i=1}^{T} T_i * FP_O[i].input$, i.e., we multiply each selected cornerstone fingerprint's input with a corresponding number and adds them all as the new input; For the output, according to Equation 2, the associativity property implies that the corresponding output for a group of associated operators has the same transformation as the input, thus we can directly get the expected output for FP_{New} as $\sum_{i=1}^{T} T_i * FP_O[i].output$, without needing to conduct one costly CPU inference for the given operator O.

With these oblivious fingerprints, SOTER sends fingerprint input to check a partitioned operator's integrity whenever a kernel switch (from enclave to GPU) happens. Specifically, when a real inference intermediate result (IR) is about to send to a partitioned operator O, SOTER randomly selects a newly derived fingerprint FP_O and sends FP_O along with the IR to run in GPU simultaneously.

D. Proof of fingerprints' obliviousness

Recall that SOTER skips the top-W operators when partitions a model (§IV-A), hence the adversary outside enclave can not determine the IRs for a given inference query because the first several operators are hidden. Owing to that, the adversary cannot determine whether an IR sent from enclave belongs to the real query input. Further, if we prove that the fingerprints (sent along with the real IRs) are oblivious, we ensure the adversary outside enclave can not bypass our integrity checking. Next, we prove the obliviousness of SOTER's dynamically derived fingerprints by analyzing the distribution of randomly sampled fingerprints and real IRs to a given partitioned operator O.

Inspired from previous work [31], we analyze the l_2 distance of two randomly sampled inputs (including both fingerprint input and real IR) sent to O, which should fits a normal (Gaussian) distribution [52], [61]. This is because, the intermediate results produced by a model are typically defined in a bounded space, i.e., elements of inputs are defined over a certain range of values. Since we expect client's queries are benign (§III-B), the input to O should be distributed in a natural and consistent manner. Thus, we expect the l_2 distance between any two randomly sampled inputs to O to be close to a normal distribution.

Figure 7 depicts the difference in the distribution of distances between oblivious fingerprints and fixed fingerprints. We observed that by using the associativity property to dynamically derive new fingerprints at runtime, the adversary can not distinguish the fingerprint data flow because the distance distribution between two random inputs is close to a normal distribution. Whereas, if we use fixed fingerprints, the adversary outside enclave can observe a totally different distribution, which implies that fingerprint data flow exists. Thus, the adversary may stop attacks and evade our detection.

V. EVALUATION

Testbed. We conducted all evaluation in our cluster on a GPU farm of 10 DELL R430 servers with 20 Nvidia RTX2080TI graphic cards. Each graphic card is connected to the host with PCIe 3.0. Each server is equipped with an Intel 2.60GHz E5-2690 V3 CPU, 64GB memory, and 40Gbps NIC. We ran SGX with the real Intel SGX hardware in our cluster.

Model description. As shown in Table II, we evaluated nine popular DNN models include a Multi-layer Perception (MLP) model, six Convolution Neural Network (CNN) models including AlexNet [35], DenseNet-121 [28], VGG-19 [59], ResNet-101 [26], NASNet [75] and PNASNet [41], a Recurrent Neural Network (RNN) WaveRNN [33] and a transformer model [2]. Among these models, the MLP model is purely composed by the same linear operators (*torch.nn.Linear*) and ReLU activation functions (*torch.nn.ReLU*); For the CNN models, AlexNet, DenseNet-121, VGG-19, and ResNet-101 are popular models that achieve high inference accuracy on computer vision tasks. In addition to these CNN models, we evaluated two large CNN models NASNet and PNASNet which are designed by recent *neural architecture search* (NAS) techniques [48]; WaveRNN is an efficient RNN model developed by Deepmind and Google for efficient speech-to-text transformations [33]; Transformer is the most popular model for natural language processing [2].

Datasets description. The parameters of the models were trained by using well-known datasets on PyTorch. We used ImageNet [11] for CV models (Image Classification tasks) and WMT16 [1] for NLP models (Translation tasks). ImageNet is a full-scale image dataset that contains 600k images from 1k classes, which has been widely used in many computer vision (CV) studies; WMT is the major translation dataset that has been used in recent natural language processing (NLP) studies.

Baselines. We compared SOTER with MLcapsule [25], a popular SGX-based secure inference system at the client-side. MLcapsule shields the entire model in SGX and executes inference tasks only with CPU, providing a fully black-box guarantee to offloaded models but coming at a substantial cost on inference latency. Since MLcapsule is not open-sourced, we built a proof-of-concept implementation based on the descriptions in MLcapsule's paper [25]. We also compared SOTER with the state-of-the-art model partition approach eNNclave [54], a notable open-sourced inference system at the

TABLE II: Overview of evaluated models and datasets.

Task	Pattern	Model	Param Size (MB)	Defaulte Input Tensor Shape	Dataset
Image Classification	MLP	MNIST-MLP (MLP) [37]	4.58	(1, 1, 28, 28)	MNIST [36]
	CNN	AlexNet (AN) [35]	233.08	(1, 3, 224, 224)	ImageNet [11]
		DenseNet-121 (DN) [28]	30.44		
		VGG-19 (VGG) [59]	548.05		
		ResNet-101 (RN) [26]	169.94		
		NASNet (NN) [75]	338.57	(1, 3, 331, 331)	
		PNASNet (PNN) [41]	328.68		
Translation	RNN	WaveRNN (WR) [33]	1254.64	512 tokens	WMT16 (En-De) [1]
	Transformer	Nvidia-Transformer (TF) [2]	1335.09	512 tokens	WMT16 (En-De) [1]

client-side that uses handcrafted model partition. Although DeepEnclave [23] uses SGX for model partition, we did not evaluate it because DeepEnclave has an orthogonal focus on protecting client's input data.

We divide our evaluation into three sections that answer the following questions:

§V-A How efficient is SOTER compared with existing secure inference systems at the client-side?

§V-B How accurate is SOTER compared with existing secure inference systems at the client-side?

§V-C How robust is SOTER under integrity attacks?

A. End-to-end Performance

End-to-end inference latency. We first investigated the end-to-end inference latency of SOTER and two baseline systems with nine models. All reported measurements are the average inference latency of 20 independent inferences on the same input. As shown in Figure 8, the partition-based secure inference systems including SOTER (execute inference tasks with both CPU and GPU) achieved better performance than MLcapsule (executes inference tasks only with CPU). This is because the CPU is at least one order of magnitude less efficient than GPU [64] in executing the inference tasks. Among all baseline systems, SOTER achieved the lowest inference latency for all models. Moreover, the latency gaps between SOTER and baseline systems grew larger as model / input size increases (shown in Figure 9). For instance, with a small DNN model (AlexNet), SOTER achieved $\sim 3.0\%$ lower inference latency on average than existing secure inference systems. With a larger DNN model (Transformer), SOTER achieved up to 22.6% lower inference latency on average than existing secure inference systems.

Latency breakdown. To understand SOTER's low inference latency, we broke down the time taken in different phases of an inference task in SOTER, and the results are reported in Table III. It can be clearly observed that, for all models, with a 80% probability ($\theta = 0.8$) to partition an associative operator, SOTER securely offloaded most computation-expensive model layers to GPU and successfully downgraded inference latency compared to pure CPU inference, resulting in low end-to-end inference latency. Doing so requires SOTER to copy the model layer parameters from SGX to the GPU memory only in the preprocessing phase. During the inference phase, SOTER only copies intermediate results (much smaller than the model parameters) into the GPU memory, and copies the outputs back to the SGX enclave memory for each layer. Frequent memory copies between GPU and SGX may cause a substantial latency penalty. To amortize these memory copy overhead, SOTER batches several continuous operators together, and performs input/output memory copies only before and after computing the batch of operators.

TABLE III: Latency breakdown of SOTER ($\theta = 0.8$).

Models	Preprocessing (s)	Inference (ms)		
		SGX (CPU)	Kernel Switch	GPU
MLP	0.32	0.08	0.005	0.001
AlexNet	1.56	3.475	0.02	0.76
VGG-19	3.52	4.82	0.05	3.64
ResNet-101	2.29	2.30	0.06	7.10
DenseNet121	2.23	1.03	0.11	4.77
NASNet	8.23	274.43	11.21	27.71
PNASNet	9.47	293.61	12.38	29.41
WaveRNN	11.51	312.25	15.13	26.35
Transformer	12.41	334.97	15.60	34.35

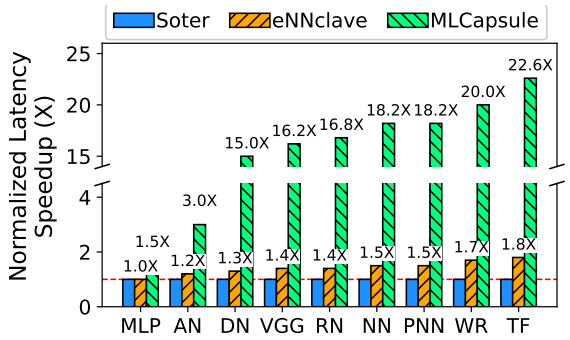


Fig. 8: Comparison of normalized end-to-end inference latency.

Overall, SOTER can achieve a low inference latency, and the computation overhead (CPU computation in SGX and kernel switch) caused by enclaving the plaintext operators only increases sub-linearly with the model size.

B. Inference Accuracy

In this section, we evaluate the resulting inference accuracy of SOTER and existing secure inference systems. For each system, we compared the inference accuracy of the protected models (i.e., running some or all layers of the ML model in SGX enclaves to protect the model) and the corresponding white-box unprotected model (i.e., running the entire model on GPU without the protection of SGX).

As shown in Figure 10, SOTER and MLcapsule achieved zero accuracy drop, while other partition-based systems caused

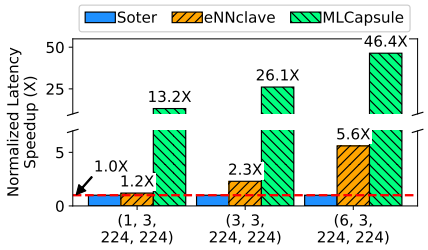


Fig. 9: Inference latency with different input shape (batch_sise, channels, height, weight).

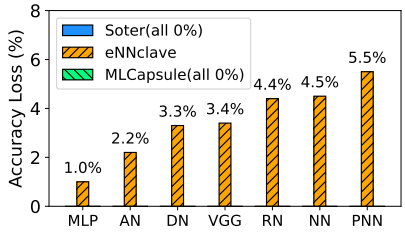


Fig. 10: Comparison of end-to-end inference accuracy.

inference accuracy drops from 1% to 5.5% for all evaluated CNN models. MLcapsule was able to achieve zero accuracy loss because it shields the entire model in the SGX enclave to protect the highest security guarantee. However, at the same time, the security guarantee of MLcapsule come at a steep price in performance, compared to untrusted alternatives (i.e., perform inference tasks on the GPU).

To achieve better performance, the partition-based secure inference systems outsource some inference computations to the untrusted GPU. However, existing partition-based inference system (i.e., eNNclave) takes a handcrafted approach to partition an offloaded model and tunes the parameters of the outsourced layers, leading to a drop in the inference accuracy. By safely morphing and partitioning computation-expensive operators to GPU for faster inference computing, and then recovers the real inference result in SGX, SOTER achieves stronger model confidentiality (§IV-B) compared to eNNclave without sacrificing the inference accuracy.

C. Robustness to Integrity Breach Attacks

We evaluated the robustness of SOTER under the integrity breach attacks. We conducted an integrity attack by continuously and randomly tempering with the outputs of layers outside the enclave, and monitoring the number of fingerprints for SOTER to detect this attack. The evaluation results of 10K attacks are shown in Figure 11. All attacks were successfully detected, and 99.1% attacks were detected with less than 10 fingerprints while we incurred zero false positives. This result matches our theoretical analysis in §IV-D.

VI. DISCUSSION

VII. CONCLUSION

This paper propose

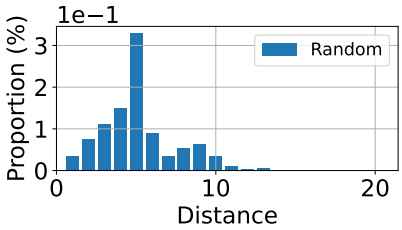


Fig. 11: Fingerprint number for catching integrity attacks.

REFERENCES

[1] "Wmt16," http://www.statmt.org/wmt16/translation-task.html, 2016.

[2] "Transformer," https://ngc.nvidia.com/catalog/resources, 2020.

[3] A. F. Agarap, "Deep learning using rectified linear units (relu)," *arXiv preprint arXiv:1803.08375*, 2018.

[4] A. Aggarwal, T. E. Carlson, R. Shokri, and S. Tople, "Soteria: In search of efficient neural networks for private inference," *arXiv preprint arXiv:2007.12934*, 2020.

[5] J. Aumasson and L. Merino, "Sgx secure enclaves in practice–security and crypto review," *Black Hat*, 2016.

[6] M. Bojarski, D. Del Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang *et al.*, "End to end learning for self-driving cars," *arXiv preprint arXiv:1604.07316*, 2016.

[7] H. Cai, L. Zhu, and S. Han, "Proxylessnas: Direct neural architecture search on target task and hardware," *arXiv preprint arXiv:1812.00332*, 2018.

[8] T. Carvalho, E. R. De Rezende, M. T. Alves, F. K. Balieiro, and R. B. Sovat, "Exposing computer generated images by eyes region classification via transfer learning of vgg19 cnn," in *2017 16th IEEE International Conference on Machine Learning and Applications (ICMLA)*. IEEE, 2017, pp. 866–870.

[9] V. Costan and S. Devadas, "Intel sgx explained." *IACR Cryptol. ePrint Arch.*, vol. 2016, no. 86, pp. 1–118, 2016.

[10] P. Das, N. Ivkin, T. Bansal, L. Rouesnel, P. Gautier, Z. Karnin, L. Dirac, L. Ramakrishnan, A. Perunicic, I. Shcherbatyi *et al.*, "Amazon sagemaker autopilot: a white box automl solution at scale," in *Proceedings of the Fourth International Workshop on Data Management for End-to-End Machine Learning*, 2020, pp. 1–7.

[11] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 2009, pp. 248–255.

[12] S. Durga, E. Daniel, and S. Deepakanmani, "Dnn-based decision support system for ecg abnormalities," in *2nd EAI International Conference on Big Data Innovation for Sustainable Cognitive Computing*. Springer, 2021, pp. 331–338.

[13] C. Dwork, "Differential privacy: A survey of results," in *International conference on theory and applications of models of computation*. Springer, 2008, pp. 1–19.

[14] T. Elsken, J. H. Metzen, and F. Hutter, "Neural architecture search: A survey," *The Journal of Machine Learning Research*, vol. 20, no. 1, pp. 1997–2017, 2019.

[15] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing, "Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy," in *International Conference on Machine Learning*. PMLR, 2016, pp. 201–210.

[16] A. Giusti, D. C. Cireşan, J. Masci, L. M. Gambardella, and J. Schmidhuber, "Fast image scanning with deep max-pooling convolutional neural networks," in *2013 IEEE International Conference on Image Processing*. IEEE, 2013, pp. 4034–4038.

[17] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT press, 2016.

[18] I. Goodfellow, P. McDaniel, and N. Papernot, "Making machine learning robust against adversarial inputs," *Communications of the ACM*, vol. 61, no. 7, pp. 56–66, 2018.

[19] Google, "Cloud speech-to-text api -language support," https://cloud.google.com/speech-to-text/docs/languages.

[20] ——, "Google lens," https://lens.google/.

[21] T. Graepel, K. Lauter, and M. Naehrig, "Ml confidential: Machine

learning on encrypted data," in *International Conference on Information Security and Cryptology*. Springer, 2012, pp. 1–21.

[22] L. Greco, P. Ritrovato, and F. Xhafa, "An edge-stream computing infrastructure for real-time analysis of wearable sensors data," *Future Generation Computer Systems*, vol. 93, pp. 515–528, 2019.

[23] Z. Gu, H. Huang, J. Zhang, D. Su, A. Lamba, D. Pendarakis, and I. Molloy, "Securing input data of deep learning inference systems via partitioned enclave execution," *arXiv preprint arXiv:1807.00969*, 2018.

[24] P. Guo, B. Hu, and W. Hu, "Mistify: Automating dnn model porting for on-device inference at the edge." in *NSDI*, 2021, pp. 705–719.

[25] L. Hanzlik, Y. Zhang, K. Grosse, A. Salem, M. Augustin, M. Backes, and M. Fritz, "Mlcapsule: Guarded offline deployment of machine learning as a service," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 3300–3309.

[26] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[27] D. A. Hensher and W. H. Greene, "The mixed logit model: the state of practice," *Transportation*, vol. 30, no. 2, pp. 133–176, 2003.

[28] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 4700–4708.

[29] J. Jaworek-Korjakowska, P. Kleczek, and M. Gorgon, "Melanoma thickness prediction based on convolutional neural network with vgg-19 model transfer learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2019, pp. 0–0.

[30] J. Jiang, X. Chen, T. Li, C. Wang, T. Shen, S. Zhao, H. Cui, C.-L. Wang, and F. Zhang, "Uranus: Simple, efficient sgx programming and its applications," in *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security*, 2020, pp. 826–840.

[31] M. Juuti, S. Szyller, S. Marchal, and N. Asokan, "Prada: protecting against dnn model stealing attacks," in *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2019, pp. 512–527.

[32] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan, "{GAZELLE}: A low latency framework for secure neural network inference," in *27th {USENIX} Security Symposium ({USENIX} Security 18)*, 2018, pp. 1651–1669.

[33] N. Kalchbrenner, E. Elsen, K. Simonyan, S. Noury, N. Casagrande, E. Lockhart, F. Stimberg, A. Oord, S. Dieleman, and K. Kavukcuoglu, "Efficient neural audio synthesis," in *International Conference on Machine Learning*. PMLR, 2018, pp. 2410–2419.

[34] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.

[35] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in neural information processing systems*, vol. 25, pp. 1097–1105, 2012.

[36] Y. LeCun, "The mnist database of handwritten digits," *http://yann. lecun. com/exdb/mnist/*, 1998.

[37] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[38] Y.-S. Lee, C.-Y. Wang, S.-F. Wang, J.-C. Wang, and C.-H. Wu, "Fully complex deep neural network for phase-incorporating monaural source separation," in *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2017, pp. 281–285.

[39] J. Li, D. Yu, J.-T. Huang, and Y. Gong, "Improving wideband speech recognition using mixed-bandwidth training data in cd-dnn-hmm," in *2012 IEEE Spoken Language Technology Workshop (SLT)*. IEEE, 2012, pp. 131–136.

[40] L. Liao, Z. Yang, Q. Liao, K. Li, K. Li, J. Liu, and Q. Tian, "A half-precision compressive sensing framework for end-to-end person re-identification," *Neural Computing and Applications*, vol. 32, no. 4, pp. 1141–1155, 2020.

[41] C. Liu, B. Zoph, M. Neumann, J. Shlens, W. Hua, L.-J. Li, L. Fei-Fei, A. Yuille, J. Huang, and K. Murphy, "Progressive neural architecture search," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 19–34.

[42] Y. Liu, X. Chen, C. Liu, and D. Song, "Delving into transferable adversarial examples and black-box attacks," *arXiv preprint arXiv:1611.02770*, 2016.

[43] X. Ma, F.-M. Guo, W. Niu, X. Lin, J. Tang, K. Ma, B. Ren, and Y. Wang, "Pconv: The missing but desirable sparsity in dnn weight pruning for real-time execution on mobile devices," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, 2020, pp. 5117–5124.

[44] L. Meng, C.-T. Lin, T.-P. Jung, and D. Wu, "White-box target attack for eeg-based bci regression problems," in *International conference on neural information processing*. Springer, 2019, pp. 476–488.

[45] P. Mishra, R. Lehmkuhl, A. Srinivasan, W. Zheng, and R. A. Popa, "Delphi: A cryptographic inference service for neural networks," in *29th {USENIX} Security Symposium ({USENIX} Security 20)*, 2020, pp. 2505–2522.

[46] M. H. Mofrad, A. Lee, and S. L. Gray, "Leveraging intel sgx to create a nondisclosure cryptographic library," *arXiv preprint arXiv:1705.04706*, 2017.

[47] R. C. Moore and W. Lewis, "Intelligent selection of language model training data," 2010.

[48] S. Nastic, T. Rausch, O. Scekic, S. Dustdar, M. Gusev, B. Koteska, M. Kostoska, B. Jakimovski, S. Ristov, and R. Prodan, "A serverless real-time data analytics platform for edge computing," *IEEE Internet Computing*, vol. 21, no. 4, pp. 64–71, 2017.

[49] D. Palossi, A. Loquercio, F. Conti, E. Flamand, D. Scaramuzza, and L. Benini, "A 64-mw dnn-based visual navigation engine for autonomous nano-drones," *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 8357–8371, 2019.

[50] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, "Practical black-box attacks against machine learning," in *Proceedings of the 2017 ACM on Asia conference on computer and communications security*, 2017, pp. 506–519.

[51] A. Paszke, A. Chaurasia, S. Kim, and E. Culurciello, "Enet: A deep neural network architecture for real-time semantic segmentation," *arXiv preprint arXiv:1606.02147*, 2016.

[52] J. Philip, *The probability distribution of the distance between two random points in a box*. KTH mathematics, Royal Institute of Technology, 2007.

[53] H. Qassim, A. Verma, and D. Feinzimer, "Compressed residual-vgg16 cnn model for big data places image recognition," in *2018 IEEE 8th Annual Computing and Communication Workshop and Conference (CCWC)*. IEEE, 2018, pp. 169–175.

[54] A. Schlögl and R. Böhme, "ennclave: Offline inference with model confidentiality," in *Proceedings of the 13th ACM Workshop on Artificial Intelligence and Security*, 2020, pp. 93–104.

[55] J. Shao and J. Zhang, "Bottlenet++: An end-to-end approach for feature compression in device-edge co-inference systems," in *2020 IEEE International Conference on Communications Workshops (ICC Workshops)*. IEEE, 2020, pp. 1–6.

[56] T. Shen, J. Jiang, Y. Jiang, X. Chen, J. Qi, S. Zhao, F. Zhang, X. Luo, and H. Cui, "Daenet: Making strong anonymity scale in a fully decentralized network," *IEEE Transactions on Dependable and Secure Computing*, 2021.

[57] Y. Shen, H. Tian, Y. Chen, K. Chen, R. Wang, Y. Xu, Y. Xia, and S. Yan, "Occlum: Secure and efficient multitasking inside a single enclave of intel sgx," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 955–970.

[58] K. Shin and T. Kuboyama, "A generalization of haussler's convolution kernel: mapping kernel," in *Proceedings of the 25th international conference on Machine learning*, 2008, pp. 944–951.

[59] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.

[60] E. L. Sonnhammer, G. Von Heijne, A. Krogh *et al.*, "A hidden markov model for predicting transmembrane helices in protein sequences." in *Ismb*, vol. 6, 1998, pp. 175–182.

[61] L. A. S. Sors and L. A. Santaló, *Integral geometry and geometric probability*. Cambridge university press, 2004.

[62] S. Tammina, "Transfer learning using vgg-16 with deep convolutional neural network for classifying images," *International Journal of Scien-*

tific and Research Publications (IJSRP), vol. 9, no. 10, pp. 143–150, 2019.

[63] S. Targ, D. Almeida, and K. Lyman, "Resnet in resnet: Generalizing residual architectures," *arXiv preprint arXiv:1603.08029*, 2016.

[64] F. Tramer and D. Boneh, "Slalom: Fast, verifiable and private execution of neural networks in trusted hardware," *arXiv preprint arXiv:1806.03287*, 2018.

[65] F. Tramèr, A. Kurakin, N. Papernot, I. Goodfellow, D. Boneh, and P. McDaniel, "Ensemble adversarial training: Attacks and defenses," *arXiv preprint arXiv:1705.07204*, 2017.

[66] T. X. Tran, M.-P. Hosseini, and D. Pompili, "Mobile edge computing: Recent efforts and five key research directions," *IEEE COMSOC MMTC Commun.-Frontiers*, 2017.

[67] C.-C. Tsai, D. E. Porter, and M. Vij, "Graphene-sgx: A practical library {OS} for unmodified applications on {SGX}," in *2017 {USENIX} Annual Technical Conference ({USENIX}{ATC} 17)*, 2017, pp. 645–658.

[68] X. Wang, Y. Han, V. C. Leung, D. Niyato, X. Yan, and X. Chen, "Convergence of edge computing and deep learning: A comprehensive survey," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 2, pp. 869–904, 2020.

[69] Y. Wang, Y.-a. Tan, W. Zhang, Y. Zhao, and X. Kuang, "An adversarial attack on dnn-based black-box object detectors," *Journal of Network and Computer Applications*, vol. 161, p. 102634, 2020.

[70] J. Won, S.-H. Seo, and E. Bertino, "A secure shuffling mechanism for white-box attack-resistant unmanned vehicles," *IEEE Transactions on Mobile Computing*, vol. 19, no. 5, pp. 1023–1039, 2019.

[71] Y. Wu, X. Qin, Y. Pan, and C. Yuan, "Convolution neural network based transfer learning for classification of flowers," in *2018 IEEE 3rd international conference on signal and image processing (ICSIP)*. IEEE, 2018, pp. 562–566.

[72] C. Zhang and P. C. Woodland, "Parameterised sigmoid and relu hidden activation functions for dnn acoustic modelling," in *Sixteenth Annual Conference of the International Speech Communication Association*, 2015.

[73] Q. Zhang, C. Xin, and H. Wu, "Gala: Greedy computation for linear algebra in privacy-preserved neural networks," *arXiv preprint arXiv:2105.01827*, 2021.

[74] Z. Zhang, H. Wang, F. Xu, and Y.-Q. Jin, "Complex-valued convolutional neural network and its application in polarimetric sar image classification," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 55, no. 12, pp. 7177–7188, 2017.

[75] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 8697–8710.

[76] P. Zuo, Y. Hua, L. Liang, X. Xie, X. Hu, and Y. Xie, "Sealing neural network models in secure deep learning accelerators," *arXiv preprint arXiv:2008.03752*, 2020.